

CS787: GENERATIVE ARTIFICIAL INTELLIGENCE

WORD EMBEDDING

Arnab Bhattacharya
arnabb@cse.iitk.ac.in

Computer Science and Engineering,
Indian Institute of Technology, Kanpur
<http://web.cse.iitk.ac.in/~cs787/>

1st semester, 2026-27

Wed 12:00–13:15 at L20, Fri 12:00–13:15 at L19

Embedding Models

- Models for *embedding* words to vector spaces

Embedding Models

- Models for *embedding* words to vector spaces
- Uses of **word vectors** (or **term vectors**)
 - Words can be compared, subtracted, found similarity of

Embedding Models

- Models for *embedding* words to vector spaces
- Uses of **word vectors** (or **term vectors**)
 - Words can be compared, subtracted, found similarity of
 - **Word similarity** task: Are two words related?
 - Arjuna:Krishna
 - **Word analogy** task: Given two related words and a third word, find the appropriate fourth word
 - Arjuna:Devadatta::Krishna:

Embedding Models

- Models for *embedding* words to vector spaces
- Uses of **word vectors** (or **term vectors**)
 - Words can be compared, subtracted, found similarity of
 - **Word similarity** task: Are two words related?
 - Arjuna:Krishna
 - **Word analogy** task: Given two related words and a third word, find the appropriate fourth word
 - Arjuna:Devadatta::Krishna:Panchajanya

Embedding Models

- Models for *embedding* words to vector spaces
- Uses of **word vectors** (or **term vectors**)
 - Words can be compared, subtracted, found similarity of
 - **Word similarity** task: Are two words related?
 - Arjuna:Krishna
 - **Word analogy** task: Given two related words and a third word, find the appropriate fourth word
 - Arjuna:Devadatta::Krishna:Panchajanya
- *Term-term context* matrix
 - **Context window**
 - Whether term i occurs within k number of terms upstream and downstream of term j

Term Co-occurrence and Context

- Global Vectors (GloVe)
- Global co-occurrence matrix X
- X_{ij} encodes how many times term i has appeared in the context of term j

Term Co-occurrence and Context

- Global Vectors (GloVe)
- Global co-occurrence matrix X
- X_{ij} encodes how many times term i has appeared in the context of term j
- Sum of a row $X_i = \sum_{\forall j} X_{ij}$ denotes the *total* number of occurrences of term i
- *Probability* that term j occurs in the context of term i is

$$P_{ij} = P(j|i) = \frac{X_{ij}}{X_i}$$

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
$k = \text{solid}$			

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
$k = \text{solid}$	1.9e-4		

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas			

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas	6.6e-5		

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas	6.6e-5	7.8e-4	

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas	6.6e-5	7.8e-4	0.084
k = water			

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas	6.6e-5	7.8e-4	0.084
k = water	3.0e-3		

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas	6.6e-5	7.8e-4	0.084
k = water	3.0e-3	2.2e-3	

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas	6.6e-5	7.8e-4	0.084
k = water	3.0e-3	2.2e-3	1.363
k = fashion			

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas	6.6e-5	7.8e-4	0.084
k = water	3.0e-3	2.2e-3	1.363
k = fashion	1.7e-5		

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas	6.6e-5	7.8e-4	0.084
k = water	3.0e-3	2.2e-3	1.363
k = fashion	1.7e-5	1.8e-5	

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas	6.6e-5	7.8e-4	0.084
k = water	3.0e-3	2.2e-3	1.363
k = fashion	1.7e-5	1.8e-5	0.944

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas	6.6e-5	7.8e-4	0.084
k = water	3.0e-3	2.2e-3	1.363
k = fashion	1.7e-5	1.8e-5	0.944

- “solid” is more relevant to “ice” than “steam”
- “gas” is more relevant to “steam” than “ice”

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas	6.6e-5	7.8e-4	0.084
k = water	3.0e-3	2.2e-3	1.363
k = fashion	1.7e-5	1.8e-5	0.944

- “solid” is more relevant to “ice” than “steam”
- “gas” is more relevant to “steam” than “ice”
- “water” is relevant to both “ice” and “steam”

Raw Probabilities and Ratio

- Raw counts X_{ij} or even raw probabilities P_{ij} may not be useful
- Depends a lot on the actual terms i and j
- Also, asymmetric
- *Ratio* is a better indicator of relevance

	$P(k ice)$	$P(k steam)$	$\frac{P(k ice)}{P(k steam)}$
k = solid	1.9e-4	2.2e-5	8.636
k = gas	6.6e-5	7.8e-4	0.084
k = water	3.0e-3	2.2e-3	1.363
k = fashion	1.7e-5	1.8e-5	0.944

- “solid” is more relevant to “ice” than “steam”
- “gas” is more relevant to “steam” than “ice”
- “water” is relevant to both “ice” and “steam”
- “fashion” is irrelevant to both “ice” and “steam”

Ratio of Probabilities

- Therefore, whether a term k is more relevant to term i than term j depends on the ratio P_{ik}/P_{jk}

Ratio of Probabilities

- Therefore, whether a term k is more relevant to term i than term j depends on the ratio P_{ik}/P_{jk}
- Hence, for *context vector* $\tilde{w}_k$ and *term vectors* w_i, w_j

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

- Vectors are of some dimensionality d

Ratio of Probabilities

- Therefore, whether a term k is more relevant to term i than term j depends on the ratio P_{ik}/P_{jk}
- Hence, for *context* vector $\tilde{w}_k$ and *term* vectors w_i, w_j

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

- Vectors are of some dimensionality d
- Since ratio of probabilities is scalar

$$F((w_i - w_j)^T \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

Ratio of Probabilities

- Therefore, whether a term k is more relevant to term i than term j depends on the ratio P_{ik}/P_{jk}
- Hence, for *context vector* $\tilde{w}_k$ and *term vectors* w_i, w_j

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

- Vectors are of some dimensionality d
- Since ratio of probabilities is scalar

$$F((w_i - w_j)^T \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

- Replacing probabilities by the same functional form

$$F((w_i - w_j)^T \tilde{w}_k) = \frac{F(w_i^T \tilde{w}_k)}{F(w_j^T \tilde{w}_k)}$$

Symmetry

- Functional form solution is $F = \exp$
- Since $F(w_i^T \tilde{w}_k) = P_{ik}$

$$w_i^T \tilde{w}_k = \log(P_{ik}) = \log(X_{ik}) - \log(X_i)$$
$$w_i^T \tilde{w}_k + b_i = \log(X_{ik})$$

Symmetry

- Functional form solution is $F = \exp$
- Since $F(w_i^T \tilde{w}_k) = P_{ik}$

$$w_i^T \tilde{w}_k = \log(P_{ik}) = \log(X_{ik}) - \log(X_i)$$
$$w_i^T \tilde{w}_k + b_i = \log(X_{ik})$$

- Bias b_i encapsulates count of term i

Symmetry

- Functional form solution is $F = \exp$
- Since $F(w_i^T \tilde{w}_k) = P_{ik}$

$$\begin{aligned}w_i^T \tilde{w}_k &= \log(P_{ik}) = \log(X_{ik}) - \log(X_i) \\w_i^T \tilde{w}_k + b_i &= \log(X_{ik})\end{aligned}$$

- Bias b_i encapsulates count of term i
- Symmetric: both terms i and k should be in the context of each other

$$w_i^T \tilde{w}_k + b_i + \tilde{b}_k = \log(X_{ik})$$

Symmetry

- Functional form solution is $F = \exp$
- Since $F(w_i^T \tilde{w}_k) = P_{ik}$

$$\begin{aligned}w_i^T \tilde{w}_k &= \log(P_{ik}) = \log(X_{ik}) - \log(X_i) \\w_i^T \tilde{w}_k + b_i &= \log(X_{ik})\end{aligned}$$

- Bias b_i encapsulates count of term i
- Symmetric: both terms i and k should be in the context of each other

$$w_i^T \tilde{w}_k + b_i + \tilde{b}_k = \log(X_{ik})$$

- To avoid zero count problems

$$w_i^T \tilde{w}_k + b_i + \tilde{b}_k = \log(1 + X_{ik})$$

Objective Function

- *Objective function* is weighted with (a function of) X_{ij}

$$\arg \min_{w_i, w_j, \dots} \sum_{i,j=1}^V f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log(1 + X_{ij}) \right)^2$$

Objective Function

- *Objective function* is weighted with (a function of) X_{ij}

$$\arg \min_{w_i, w_j, \dots} \sum_{i,j=1}^V f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log(1 + X_{ij}) \right)^2$$

- Properties of weighting function $f(X_{ij})$
 - $f(0) \rightarrow 0$
 - $f(X_{ij})$ is non-decreasing
 - $f(X_{ij})$ should not increase heavily for large values of x

Objective Function

- *Objective function* is weighted with (a function of) X_{ij}

$$\arg \min_{w_i, w_j, \dots} \sum_{i,j=1}^V f(X_{ij}) \left(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log(1 + X_{ij}) \right)^2$$

- Properties of weighting function $f(X_{ij})$
 - $f(0) \rightarrow 0$
 - $f(X_{ij})$ is non-decreasing
 - $f(X_{ij})$ should not increase heavily for large values of x
- Following function is used

$$f(X_{ij}) = \begin{cases} \left(\frac{X_{ij}}{X_{ij}^{max}} \right)^\alpha & \text{if } X_{ij} < X_{ij}^{max} \\ 1 & \text{otherwise} \end{cases}$$

- $\alpha = 3/4$
- $X_{ij}^{max} = 100$

Discussion

- Context window of size ± 5
- Dimensionality of 100 or 300

Discussion

- Context window of size ± 5
- Dimensionality of 100 or 300
- Captures *local context* and *global pairwise statistics*

Term Embedding

- Aim is to learn an embedding vector for each term that can *predict another term* in its context
- Thus, modeled as a *classification* problem

Term Embedding

- Aim is to learn an embedding vector for each term that can *predict another term* in its context
- Thus, modeled as a *classification* problem
- ANNs used: **Word2Vec** model
- If vocabulary size is V , this is a V -class classification problem

Term Embedding

- Aim is to learn an embedding vector for each term that can *predict another term* in its context
- Thus, modeled as a *classification* problem
- ANNs used: **Word2Vec** model
- If vocabulary size is V , this is a V -class classification problem
- Input is a one-hot *term vector* of dimensionality V

Term Embedding

- Aim is to learn an embedding vector for each term that can *predict another term* in its context
- Thus, modeled as a *classification* problem
- ANNs used: **Word2Vec** model
- If vocabulary size is V , this is a V -class classification problem
- Input is a one-hot *term vector* of dimensionality V
- Output layer consists of V nodes
- Only one output vector of dimensionality V

Term Embedding

- Aim is to learn an embedding vector for each term that can *predict another term* in its context
- Thus, modeled as a *classification* problem
- ANNs used: **Word2Vec** model
- If vocabulary size is V , this is a V -class classification problem
- Input is a one-hot *term vector* of dimensionality V
- Output layer consists of V nodes
- Only one output vector of dimensionality V
- **Softmax** used for classification
- Probability of a particular dimension i with value f_i is

$$p(o_i) = \frac{\exp(f_i)}{\sum_{\forall i} \exp(f_i)}$$

Term Embedding

- Aim is to learn an embedding vector for each term that can *predict another term* in its context
- Thus, modeled as a *classification* problem
- ANNs used: **Word2Vec** model
- If vocabulary size is V , this is a V -class classification problem
- Input is a one-hot *term vector* of dimensionality V
- Output layer consists of V nodes
- Only one output vector of dimensionality V
- **Softmax** used for classification
- Probability of a particular dimension i with value f_i is

$$p(o_i) = \frac{\exp(f_i)}{\sum_{\forall i} \exp(f_i)}$$

- Choose the word w whose probability $p(w)$ is the *highest*

One-Word Context

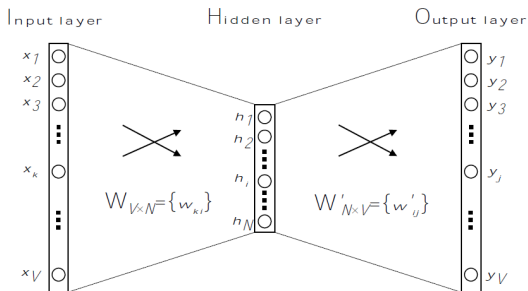
- *One* word per context is used to predict *one* target word
 - Subhas : ?

One-Word Context

- *One* word per context is used to predict *one* target word
 - Subhas : ? Bose

One-Word Context

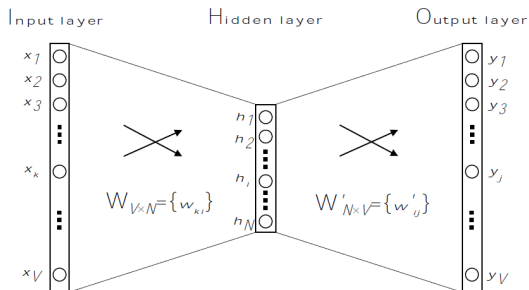
- One word per context is used to predict one target word
 - Subhas : ? Bose



- Only one hidden layer of size N
- Fully connected feed-forward architecture
- Number of weights is

One-Word Context

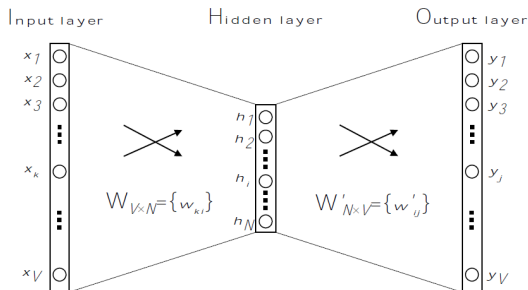
- One word per context is used to predict one target word
 - Subhas : ? Bose



- Only one hidden layer of size N
- Fully connected feed-forward architecture
- Number of weights is $V \cdot N + N \cdot V$

One-Word Context

- *One* word per context is used to predict *one* target word
 - Subhas : ? Bose



- Only one hidden layer of size N
- Fully connected feed-forward architecture
- Number of weights is $V \cdot N + N \cdot V$
- Given a one-hot encoded vector $\vec{x}_I$ for input w_I , the output is $\vec{y}_O$ predicting the word w_O

- Weight matrix W of size $V \cdot N$ from input to hidden layer
- Row k represents vector for word k : $\mathbf{v}_k^T$
- For input vector $\mathbf{x}$ of word k

$$\mathbf{h} = W^T \mathbf{x} = W^T I_{(k)} = \mathbf{v}_{w_i}^T$$

- $\mathbf{v}_{w_i}$ is the vector representation of input word W_i in N dimensions
- Essentially, the hidden layer is a *linear* copy

Layers

- Weight matrix W of size $V \cdot N$ from input to hidden layer
- Row k represents vector for word k : $\mathbf{v}_k^T$
- For input vector $\mathbf{x}$ of word k

$$\mathbf{h} = W^T \mathbf{x} = W^T I_{(k)} = \mathbf{v}_{w_i}^T$$

- $\mathbf{v}_{w_i}$ is the vector representation of input word w_i in N dimensions
- Essentially, the hidden layer is a *linear* copy
- Different weight matrix W' of size $N \cdot V$ from hidden to output layer
- Score for each word w_j is

$$u_j = \mathbf{v}_{w_j}'^T \cdot \mathbf{h} = \mathbf{v}_{w_j}'^T \cdot \mathbf{v}_{w_i}$$

- $\mathbf{v}_{w_j}'$ is the j th column of W'

Objective

- Using softmax, a multinomial distribution over all predicted words w_j given an input word w_i is defined
- *Log-linear* classification model
- Probability of target word being w_j given an input context word w_i is

$$p(w_j|w_i) = y_j = \frac{\exp(u_j)}{\sum_{j'=1}^V \exp(u_{j'})} = \frac{\exp(\mathbf{v}'_{w_j} \cdot \mathbf{v}_{w_i})}{\sum_{j'=1}^V \exp(\mathbf{v}'_{w_{j'}} \cdot \mathbf{v}_{w_i})}$$

- $\mathbf{v}_w$ is the *input vector* of word w
- $\mathbf{v}'_w$ is the *output vector* of word w

Objective

- Using softmax, a multinomial distribution over all predicted words w_j given an input word w_i is defined
- *Log-linear* classification model
- Probability of target word being w_j given an input context word w_i is

$$p(w_j|w_i) = y_j = \frac{\exp(u_j)}{\sum_{j'=1}^V \exp(u_{j'})} = \frac{\exp(\mathbf{v}'_{w_j} \cdot \mathbf{v}_{w_i})}{\sum_{j'=1}^V \exp(\mathbf{v}'_{w_{j'}} \cdot \mathbf{v}_{w_i})}$$

- $\mathbf{v}_w$ is the *input vector* of word w
- $\mathbf{v}'_w$ is the *output vector* of word w
- Training objective is to *maximize* the probabilities

$$\max p(w_{j_*}|w_i) = \max y_{j_*} = \max \log y_{j_*} = u_{j_*} - \log \sum_{j'=1}^V \exp(u_{j'}) = -E$$

Objective

- Using softmax, a multinomial distribution over all predicted words w_j given an input word w_i is defined
- *Log-linear* classification model
- Probability of target word being w_j given an input context word w_i is

$$p(w_j|w_i) = y_j = \frac{\exp(u_j)}{\sum_{j'=1}^V \exp(u_{j'})} = \frac{\exp(\mathbf{v}'_{w_j} \cdot \mathbf{v}_{w_i})}{\sum_{j'=1}^V \exp(\mathbf{v}'_{w_{j'}} \cdot \mathbf{v}_{w_i})}$$

- $\mathbf{v}_w$ is the *input* vector of word w
- $\mathbf{v}'_w$ is the *output* vector of word w
- Training objective is to *maximize* the probabilities

$$\max p(w_{j_*}|w_i) = \max y_{j_*} = \max \log y_{j_*} = u_{j_*} - \log \sum_{j'=1}^V \exp(u_{j'}) = -E$$

- Which is the word vector representation?

Objective

- Using softmax, a multinomial distribution over all predicted words w_j given an input word w_i is defined
- *Log-linear* classification model
- Probability of target word being w_j given an input context word w_i is

$$p(w_j|w_i) = y_j = \frac{\exp(u_j)}{\sum_{j'=1}^V \exp(u_{j'})} = \frac{\exp(\mathbf{v}'_{w_j} \cdot \mathbf{v}_{w_i})}{\sum_{j'=1}^V \exp(\mathbf{v}'_{w_{j'}} \cdot \mathbf{v}_{w_i})}$$

- $\mathbf{v}_w$ is the *input vector* of word w
- $\mathbf{v}'_w$ is the *output vector* of word w
- Training objective is to *maximize* the probabilities

$$\max p(w_{j_*}|w_i) = \max y_{j_*} = \max \log y_{j_*} = u_{j_*} - \log \sum_{j'=1}^V \exp(u_{j'}) = -E$$

- Which is the word vector representation?
- *Hidden layer* of dimensionality N

Objective

- Using softmax, a multinomial distribution over all predicted words w_j given an input word w_i is defined
- *Log-linear* classification model
- Probability of target word being w_j given an input context word w_i is

$$p(w_j|w_i) = y_j = \frac{\exp(u_j)}{\sum_{j'=1}^V \exp(u_{j'})} = \frac{\exp(\mathbf{v}'_{w_j} \cdot \mathbf{v}_{w_i})}{\sum_{j'=1}^V \exp(\mathbf{v}'_{w_{j'}} \cdot \mathbf{v}_{w_i})}$$

- $\mathbf{v}_w$ is the *input* vector of word w
- $\mathbf{v}'_w$ is the *output* vector of word w
- Training objective is to *maximize* the probabilities

$$\max p(w_{j_*}|w_i) = \max y_{j_*} = \max \log y_{j_*} = u_{j_*} - \log \sum_{j'=1}^V \exp(u_{j'}) = -E$$

- Which is the word vector representation?
- *Hidden layer* of dimensionality N
- The output layer is ultimately thrown away!

Continuous Bag-of-Words (CBOW) Model

- **CBOW**: *Multiple context words* to predict a *single target word*
 - Tendulkar, Dravid, Laxman : ?

Continuous Bag-of-Words (CBOW) Model

- **CBOW**: *Multiple context words* to predict a *single target word*
 - Tendulkar, Dravid, Laxman : ? Ganguly

Continuous Bag-of-Words (CBOW) Model

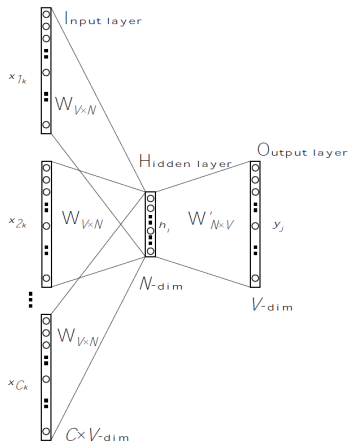
- **CBOW**: *Multiple context words* to predict a *single target word*
 - Tendulkar, Dravid, Laxman : ? Ganguly
 - Sachin, Rahul : ?

Continuous Bag-of-Words (CBOW) Model

- **CBOW**: *Multiple context words* to predict a *single target word*
 - Tendulkar, Dravid, Laxman : ? Ganguly
 - Sachin, Rahul : ? DevVarman

Continuous Bag-of-Words (CBOW) Model

- **CBOW**: *Multiple context words* to predict a *single target word*
 - Tendulkar, Dravid, Laxman : ? Ganguly
 - Sachin, Rahul : ? DevVarman



- Bag-of-words in a local context window that continuously changes

- Instead of just a copy, hidden node is average of C context words

$$\mathbf{h} = \frac{1}{C} W^T (\mathbf{x}_1 + \mathbf{x}_2 + \cdots + \mathbf{x}_C) = \frac{1}{C} (\mathbf{v}_{w_1} + \mathbf{v}_{w_2} + \cdots + \mathbf{v}_{w_C})^T$$

- Instead of just a copy, hidden node is average of C context words

$$\mathbf{h} = \frac{1}{C} W^T (\mathbf{x}_1 + \mathbf{x}_2 + \cdots + \mathbf{x}_C) = \frac{1}{C} (\mathbf{v}_{w_1} + \mathbf{v}_{w_2} + \cdots + \mathbf{v}_{w_C})^T$$

- Error

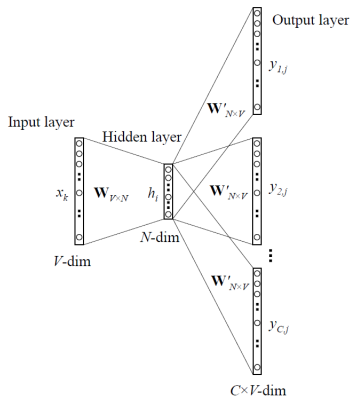
$$E = -\log p(w_{j*} | w_{l,1}, \dots, w_{l,C}) = -u_{j*} + \log \sum_{j'=1}^V \exp(u'_{j'})$$

Skip-Gram Model

- **Skip-gram**: *Single context word to predict multiple target words*
 - DevVarman : ? , ?

Skip-Gram Model

- **Skip-gram**: *Single context word to predict multiple target words*
 - DevVarman : ? , ? Sachin, Rahul



- Output layer is C target words of dimensionality V

Model

- Output layer weights are shared, i.e., they are the same matrix W'
- Probability of c^{th} target word being $w_{c,j}$ given an input context word w_I is

$$p(w_{c,j}|w_I) = y_{c,j} = \frac{\exp(u_{c,j})}{\sum_{j'=1}^V \exp(u_{j'})} = \frac{\exp(\mathbf{v}'_{w_j} \cdot \mathbf{v}_{w_I})}{\sum_{j'=1}^V \exp(\mathbf{v}'_{w_{j'}} \cdot \mathbf{v}_{w_I})}$$

Model

- Output layer weights are shared, i.e., they are the same matrix W'
- Probability of c^{th} target word being $w_{c,j}$ given an input context word w_l is

$$p(w_{c,j}|w_l) = y_{c,j} = \frac{\exp(u_{c,j})}{\sum_{j'=1}^V \exp(u_{j'})} = \frac{\exp(\mathbf{v}'_{w_j} \cdot \mathbf{v}_{w_l})}{\sum_{j'=1}^V \exp(\mathbf{v}'_{w_{j'}} \cdot \mathbf{v}_{w_l})}$$

- Error

$$E = -\log p(w_{j^*,1}, \dots, w_{j^*,c}|w_l) = -\log \prod_{c=1}^C \frac{\exp(u_{c,j^*_c})}{\sum_{j'=1}^V \exp(u_{j'})}$$

Word2Vec Discussion

- Context window size used is 4 words up and down

Word2Vec Discussion

- Context window size used is 4 words up and down
- Note that, k is chosen randomly within 4
 - This is why it is called “continuous” bag-of-words

Word2Vec Discussion

- Context window size used is 4 words up and down
- Note that, k is chosen randomly within 4
 - This is why it is called “continuous” bag-of-words
- Number of context words, C , is within 5

Word2Vec Discussion

- Context window size used is 4 words up and down
- Note that, k is chosen randomly within 4
 - This is why it is called “continuous” bag-of-words
- Number of context words, C , is within 5
- Rare words are discarded
 - This also effectively increases context window size

Word2Vec Discussion

- Context window size used is 4 words up and down
- Note that, k is chosen randomly within 4
 - This is why it is called “continuous” bag-of-words
- Number of context words, C , is within 5
- Rare words are discarded
 - This also effectively increases context window size
- Log-linear model

Word2Vec Discussion

- Context window size used is 4 words up and down
- Note that, k is chosen randomly within 4
 - This is why it is called “continuous” bag-of-words
- Number of context words, C , is within 5
- Rare words are discarded
 - This also effectively increases context window size
- Log-linear model
 - This makes training faster

Word2Vec Discussion

- Context window size used is 4 words up and down
- Note that, k is chosen randomly within 4
 - This is why it is called “continuous” bag-of-words
- Number of context words, C , is within 5
- Rare words are discarded
 - This also effectively increases context window size
- Log-linear model
 - This makes training faster
- Input word and context word are treated differently

Word2Vec Discussion

- Context window size used is 4 words up and down
- Note that, k is chosen randomly within 4
 - This is why it is called “continuous” bag-of-words
- Number of context words, C , is within 5
- Rare words are discarded
 - This also effectively increases context window size
- Log-linear model
 - This makes training faster
- Input word and context word are treated differently
 - Two roles and mathematical convenience

Word2Vec Discussion

- Context window size used is 4 words up and down
- Note that, k is chosen randomly within 4
 - This is why it is called “continuous” bag-of-words
- Number of context words, C , is within 5
- Rare words are discarded
 - This also effectively increases context window size
- Log-linear model
 - This makes training faster
- Input word and context word are treated differently
 - Two roles and mathematical convenience
- *Corpus* is more important than method

Word2Vec Discussion

- Context window size used is 4 words up and down
- Note that, k is chosen randomly within 4
 - This is why it is called “continuous” bag-of-words
- Number of context words, C , is within 5
- Rare words are discarded
 - This also effectively increases context window size
- Log-linear model
 - This makes training faster
- Input word and context word are treated differently
 - Two roles and mathematical convenience
- *Corpus* is more important than method
- Why just context?

Word2Vec Discussion

- Context window size used is 4 words up and down
- Note that, k is chosen randomly within 4
 - This is why it is called “continuous” bag-of-words
- Number of context words, C , is within 5
- Rare words are discarded
 - This also effectively increases context window size
- Log-linear model
 - This makes training faster
- Input word and context word are treated differently
 - Two roles and mathematical convenience
- *Corpus* is more important than method
- Why just context?
 - No definitive or satisfying answer

Morphologically Richer Languages

- Morphologically richer languages have the same internal root or structure for a large number of words
- Further, it requires a prohibitively large corpus to train for all such variants of a word
- Examples: Indian languages, Finnish, Turkish, and even European languages such as French, Spanish, German, Russian

Morphologically Richer Languages

- Morphologically richer languages have the same internal root or structure for a large number of words
- Further, it requires a prohibitively large corpus to train for all such variants of a word
- Examples: Indian languages, Finnish, Turkish, and even European languages such as French, Spanish, German, Russian
- In Indian languages, compound words are a problem too

Morphologically Richer Languages

- Morphologically richer languages have the same internal root or structure for a large number of words
- Further, it requires a prohibitively large corpus to train for all such variants of a word
- Examples: Indian languages, Finnish, Turkish, and even European languages such as French, Spanish, German, Russian
- In Indian languages, compound words are a problem too
- Simple corpus-based word vector embeddings may not work

Morphologically Richer Languages

- Morphologically richer languages have the same internal root or structure for a large number of words
- Further, it requires a prohibitively large corpus to train for all such variants of a word
- Examples: Indian languages, Finnish, Turkish, and even European languages such as French, Spanish, German, Russian
- In Indian languages, compound words are a problem too
- Simple corpus-based word vector embeddings may not work
- Byte-pair encodings, etc. alleviate the problem

Morphologically Richer Languages

- Morphologically richer languages have the same internal root or structure for a large number of words
- Further, it requires a prohibitively large corpus to train for all such variants of a word
- Examples: Indian languages, Finnish, Turkish, and even European languages such as French, Spanish, German, Russian
- In Indian languages, compound words are a problem too
- Simple corpus-based word vector embeddings may not work
- Byte-pair encodings, etc. alleviate the problem
- **FastText** uses **character n-grams**
- Originally called **SISG (Subword Information Skip Gram)**

- A word vector representation is *sum* of character n-gram vectors and the word itself

- A word vector representation is *sum* of character n-gram vectors and the word itself
- Two limits for n-gram length: minimum and maximum
 - Generally, 3 and 6

- A word vector representation is *sum* of character n-gram vectors and the word itself
- Two limits for n-gram length: minimum and maximum
 - Generally, 3 and 6
- Example word: “India”
- If limits are 2 and 3, then embeddings are of

- A word vector representation is *sum* of character n-gram vectors and the word itself
- Two limits for n-gram length: minimum and maximum
 - Generally, 3 and 6
- Example word: “India”
- If limits are 2 and 3, then embeddings are of “In”, “Ind”, “ndi”, “dia”, “ia”, and “India”

- A word vector representation is *sum* of character n-gram vectors and the word itself
- Two limits for n-gram length: minimum and maximum
 - Generally, 3 and 6
- Example word: “India”
- If limits are 2 and 3, then embeddings are of “In”, “Ind”, “ndi”, “dia”, “ia”, and “India”
- If G_w is the set of n-grams for a word w , then

$$s(w, c) = \sum_{\forall g \in G_w} \mathbf{z}_g^T \cdot \mathbf{v}_c$$

- A word vector representation is *sum* of character n-gram vectors and the word itself
- Two limits for n-gram length: minimum and maximum
 - Generally, 3 and 6
- Example word: “India”
- If limits are 2 and 3, then embeddings are of “In”, “Ind”, “ndi”, “dia”, “ia”, and “India”
- If G_w is the set of n-grams for a word w , then

$$s(w, c) = \sum_{\forall g \in G_w} \mathbf{z}_g^T \cdot \mathbf{v}_c$$

- FastText can handle *out-of-vocabulary* words
 - Glove and Word2Vec cannot!
- Uses as many common n-grams as it can from the new word